



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

ICN-292 – Sistemas de Información para la Gestión

Laboratorio 3

Automatización de procesos con n8n

<https://github.com/trinidad-usm/ICN292-Lab3-Salinas-Trinidad>

Profesor: José Luis Sáez Tamayo

Nombre: Trinidad Salinas

S: 964

Resumen Ejecutivo

Semilla personal: S = 964 | Parámetros aplicados: U = 44.000 (umbral de monto), D = 7 días (plazo máximo).

Para este laboratorio se construyó en n8n un producto mínimo viable que automatiza el triage de solicitudes de devolución de AndesHogar SpA. Se armaron 3 Workflows: uno principal de triage (Parte A), uno de resumen diario programado (Parte B) y uno emisor para probar el sistema con las 15 solicitudes de prueba.

El flujo de triage recibe cada solicitud por un Webhook, la evalúa contra 6 reglas de negocio ordenadas por precedencia mediante un nodo Switch y según el resultado la clasifica en una de 5 rutas: INVALIDO, RECHAZO, REVISION o APROBACION. Cada ruta consulta el valor de la UF en la API publica mindicador.cl, notifica al cliente, registra la solicitud con su ruta, y responde. El flujo de resumen lee ese registro todos los días a las 18:00 hrs y consolida cuantas solicitudes se procesaron, el monto total del día y la tasa de aprobación.

Las 4 rutas del flujo triage se probaron con datos reales y coincidieron con lo esperado en todos los casos. La API de mindicador.cl mostro ser inestable durante las pruebas, por lo que se configuró Timeout, Retry On Fail y On Error en modo “Continue” en los 4 nodos HTTP Request, para que el flujo no se detenga por completo si el servicio externo falla. Para el registro se optó por una Data Table en vez de Google Sheets, ya que no requiere credenciales externas y permite que esos mismos datos se reutilicen en el flujo de resumen de forma persistente.

En la Parte C se documentaron 3 entradas invalidas (mostrando que el flujo maneja bien los campos vacíos, pero no los valores no convertibles a número), se explicó el uso de On Error y de un Error Workflow, se comprobó que las comparaciones de monto y plazo son inclusivas mediante 2 casos de frontera, también se demostró el efecto de cambiar el tipo de una condición de Number a String y finalmente se identificaron 2 casos que el flujo actual no puede resolver (montos escritos en palabras y SKUs inexistentes), señalando qué se necesitaría para detectarlos.

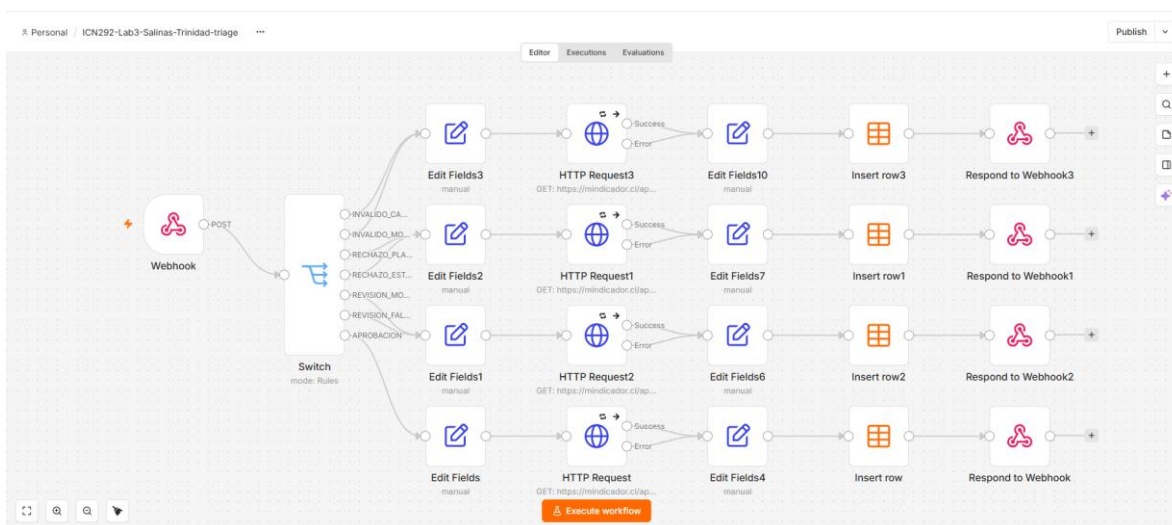
En base a lo anterior, se recomienda que este MVP no se convierta directamente en el sistema definitivo sin antes agregar una validación de tipo de dato antes del Switch y una verificación del SKU contra el catálogo real de productos, ya que ambos puntos son las principales debilidades detectadas durante las pruebas.

Parte A: Flujo de triage

El flujo principal está compuesto por 8 nodos que se ramifican en 4 caminos, según el resultado de la evaluación de reglas: **INVALIDO**, **RECHAZO**, **REVISION** y **APROBACION**. La estructura completa es:

Webhook → Switch (6 reglas + fallback) → [Edit Fields por ruta] → HTTP Request (UF) → Edit Fields (notificación) → Data Table (registro) → Respond to Webhook

Esta misma cadena de 5 pasos se repite una vez por cada una de las 4 rutas, por lo que el flujo termina teniendo 24 nodos en total además del Webhook y el Switch.



1. El nodo Webhook

Se configuró en método POST. También, se cambió su parámetro “Respond” a “Using Respond to Webhook Node”, con el fin de que la respuesta se genere al final del flujo (con el nodo Respond to Webhook) y no automáticamente al recibir la solicitud.

2. El nodo Switch: las 6 reglas de negocio

Se configuró en modo Rules, con las 6 reglas en el mismo orden de precedencia que exige la guía del laboratorio (la primera que se cumpla gana y detiene la evaluación):

#	Condición	Salida
1	id_solicitud está vacío	INVALIDO CAMPO
2	monto <= 0	INVALIDO MONTO
3	días_desde_compra > 7	RECHAZO PLAZO
4	estado_producto = danado_por_uso	RECHAZO ESTADO
5	monto > 44.000	REVISION MONTO
6	estado_producto = con_fallas	REVISION FALLA
-	ninguna de las anteriores	APROBACION

Se utilizó `.toNumber()` en las condiciones sobre monto y días, ya que estos campos llegan como texto desde el formulario. Las comparaciones quedaron con `>` estricto, para que un valor exactamente igual a U o D no caiga en rechazo o revisión, tal como pide la regla de comparaciones inclusivas de la guía. Para la rama de aprobación se usó el “Extra Output” del Switch, que es la forma en que esta versión del nodo maneja el fallback.

3. Los 4 Edit Fields de ruta y motivo

Las 6 salidas del Switch se agrupan en solo 4 nodos Edit Fields, cada uno escribiendo dos campos de texto fijo: ruta y motivo.

4. HTTP Request a mindicador.cl

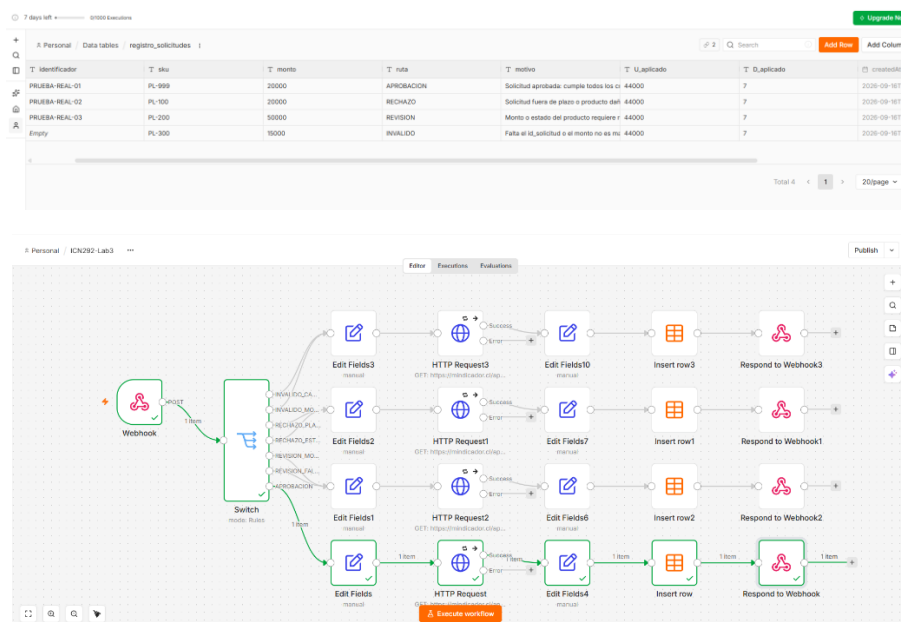
Cada rama tiene su propio HTTP Request, en su método GET, apuntando a <https://mindicador.cl/api/uf>, sin autenticación. Durante las pruebas esta API se mostró bastante inestable (se quedaba pegada o se caía la conexión), así que en la pestaña Setting de cada uno de los 4 nodos se configuró un Timeout de 10 segundos, se activó Retry On Fail y se puso On Error en “Continue (using error output)”. De esta manera, aunque la API falle, el flujo sigue avanzando en vez de detenerse. Las dos salidas del nodo (Success y Error) se conectaron al mismo nodo siguiente.

5. Notificación y registro

La notificación al cliente se simuló con un Edit Fields con un texto, ya que no se posee las credenciales de correo o SMS configuradas. Para el registro se usó una Data Table (registro_solicitudes) con la operación Insert Row, en vez de solo un Edit Fields, porque se necesitaba que esos datos quedaran guardados de forma constante para poder leerlos después en el Workflow de la Parte B. Cada fila guarda: identificador, sku, monto, ruta, motivo, U_aplicado y D_aplicado.

6. Pruebas realizadas

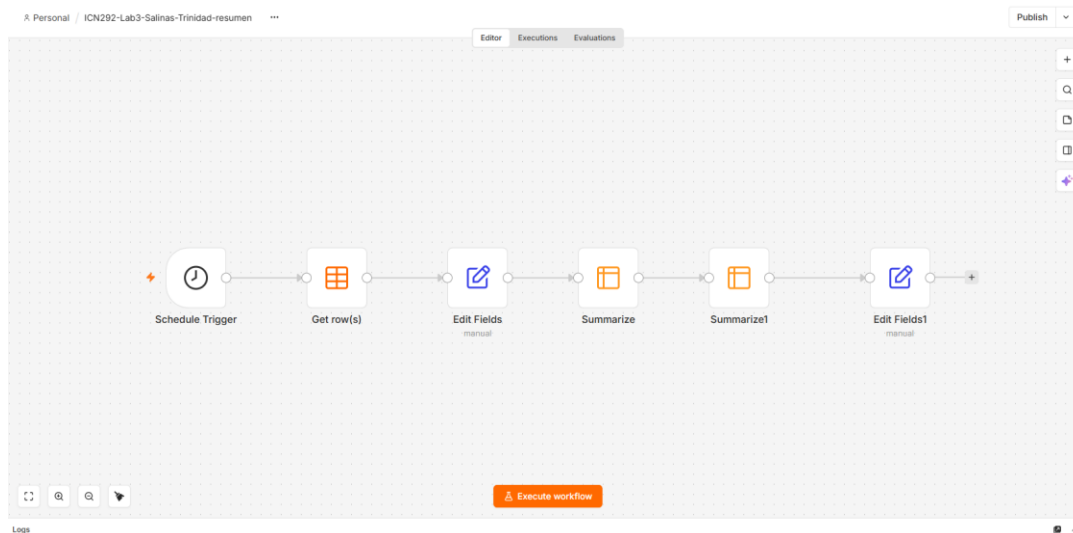
Se realizaron 4 pruebas de ruta mandando solicitudes reales al Webhook, usando primero reqbin y después Hoppscotch, y en los 4 casos la ruta obtenida coincidió con lo esperado según los datos enviados.



Parte B: Flujo programado de resumen

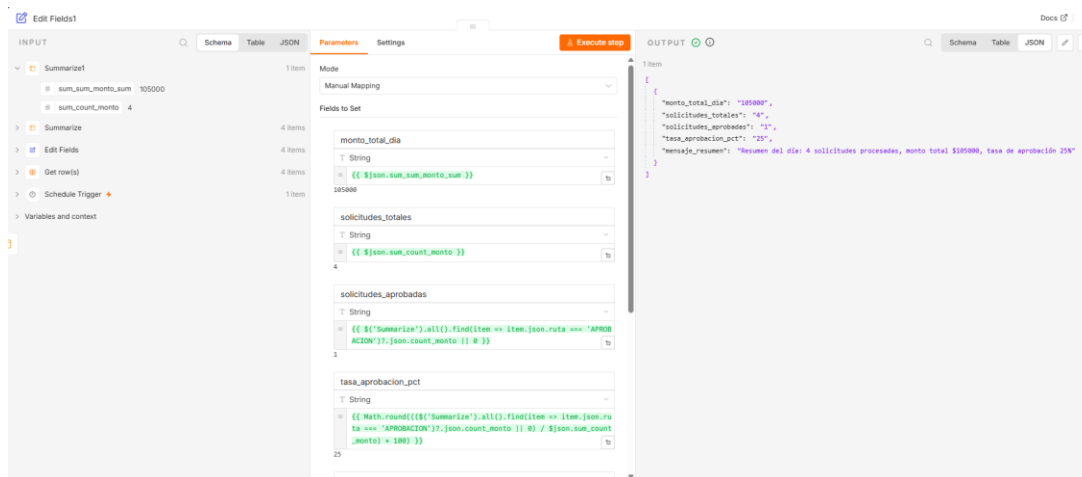
Este Workflow usa un Schedule Trigger configurado para dispararse todos los días a las 18:00 hrs, simulando el cierre de jornada del equipo de Servicio al Cliente. La secuencia es:

Schedule Trigger → Gest Row(s) (lee toda la tabla de registro) → Edit Fields (convierte el monto de texto a número, porque en la tabla queda guardado como texto) → Summarize agrupado por ruta (suma y cuenta por cada ruta) → Summarize general (suma los subtotales anteriores, sin agrupar) → Edit Fields final (calcula la tasa de aprobacion y arma el mensaje)



Se conectaron los dos Summarize en cadena, uno después del otro, en vez de en paralelo porque así se podía usar en el Edit Fields final una referencia directa al primer Summarize para buscar específicamente la fila de APORBACION y sacar cuantas solicitudes fueron aprobadas, sin necesitar un nodo Merge.

El mensaje final que se arma queda algo así: “Resumen del día: 4 solicitudes procesadas, monto total \$105.000, tasa de aprobación 25%”.



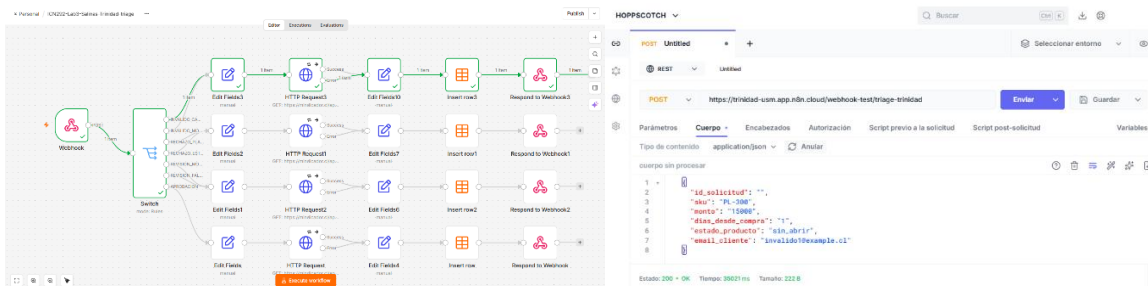
Si un día no se procesa ninguna solicitud, el resumen saldría igual pero con todos los valores en 0.

Parte C: Errores y casos borde

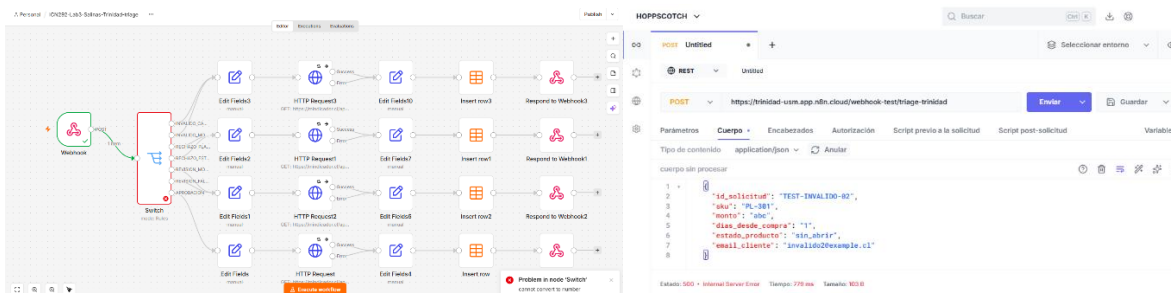
1. Tres entradas invalidas

Se probaron 3 solicitudes distintas para ver como reaccionaba el flujo ante datos erróneos:

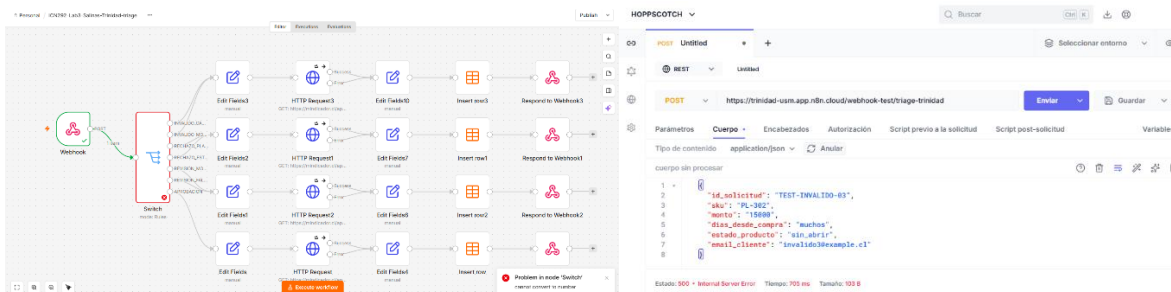
Caso 1 – Campo Vacío: Se mandó una solicitud con *id_solicitud*: “ ”. Como se esperaba, cayó en INVALIDO_CAMPO, respuesta 200 OK y el flujo completo se vio en verde.



Caso 2 – Monto no numérico: Se mando una solicitud con *monto*: “abc”. Se esperaba que cayera en INVALIDO_MONTO, pero en vez de eso el nodo Switch falló directamente con el error “Cannot convert to number” y la respuesta fue un 500 Internal Server Error. El flujo no llegó a clasificar la solicitud, se detuvo en ese mismo punto.



Caso 3 – Días no numéricos: Se mandó una solicitud con *días_desde_compra*: “muchos” y pasó exactamente lo mismo que en el caso 2: el Switch falló con el mismo error de conversión y la respuesta también fue un 500.



2. On Error en cada nodo

On Error es una configuración que tiene cada nodo por separado y define que pasa si ese nodo falla durante la ejecución. Por defecto todos los nodos vienen en “Stop Workflow” (se detiene la ejecución). La otra opción que se utilizó en este caso fue “Continue (using error output)”, que deja que el flujo siga, aunque ese nodo fallé. Lo configurado fue:

- **Los 4 HTTP Request** (a mindicator.cl): se dejaron en “Continue (using error output)”, porque esa API se cayó o tardó mucho varias veces durante las pruebas. No tiene sentido que todo el flujo se detuviera solo porque no llegó el dato de la UF, ya que el resto (notificar, registrar, responder) puede seguir funcionando igual
- **Todos los demás nodos** (Switch, Webhook, Edit Fields, Data Table, Respond to Webhook): se dejaron en “Stop Workflow”. Para el Switch en particular, esto tiene sentido porque si no puede evaluar bien las reglas se prefiere que detenga en vez de inventar una clasificación equivocada.

3. Que es un Error Workflow

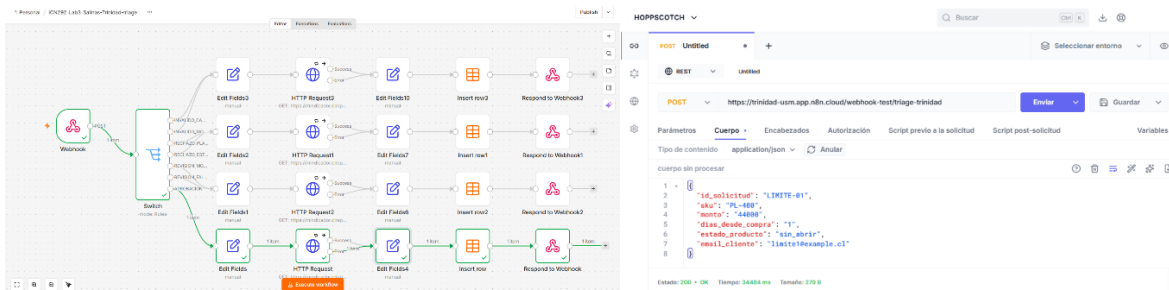
Un Error Workflow es un Workflow aparte que n8n puede disparar automáticamente cuando cualquier ejecución de otro Workflow termina en error. En este caso no se configuro ninguno.

Esto es importante sobre todo para el flujo de la Parte B porque se dispara solo, todos los días a las 18:00, sin que nadie este mirando en ese momento. Si algún nodo de ese flujo fallara, como no existe Error Workflow, nadie se enteraría: el resumen simplemente no se generaría esa noche y no quedaría ningún registro ni alerta de que algo salió mal. Es distinto al flujo de triage, donde al menos el error 500 le llega a quien mando la solicitud.

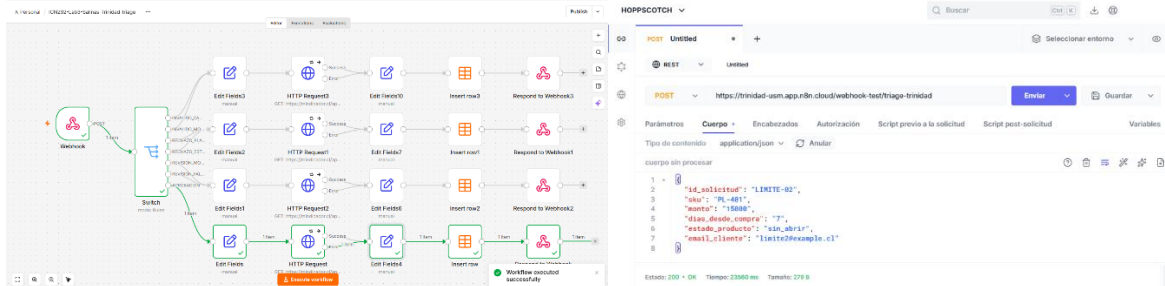
4. Dos casos frontera

Se probaron 2 solicitudes justo en el límite de las reglas establecidas, para confirmar que las comparaciones son inclusivas.

Caso límite 1 – monto = U exacto (44.000): Se mandó una solicitud con *monto*: “44000”. Como la regla 5 usa > estricto, un monto igual a U no debería activar REVISION_MONTO, lo que se comprobó cuando efectivamente cayó en APROBACION.



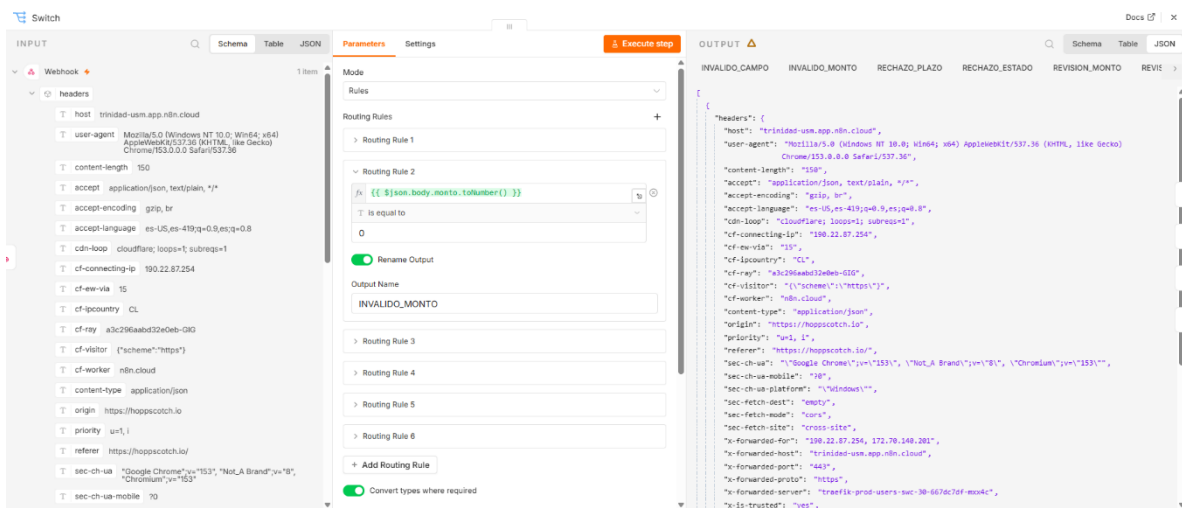
Caso límite 2 – días = D exacto (7): Se mandó una solicitud con *días_desde_compra*: “7”. De la misma manera que el caso anterior, la regla 3 usa > estricto, así que 7 días exactos no deberían activar RECHAZO_PLAZO, lo que también se comprobó cuando cayó en APROBACION.



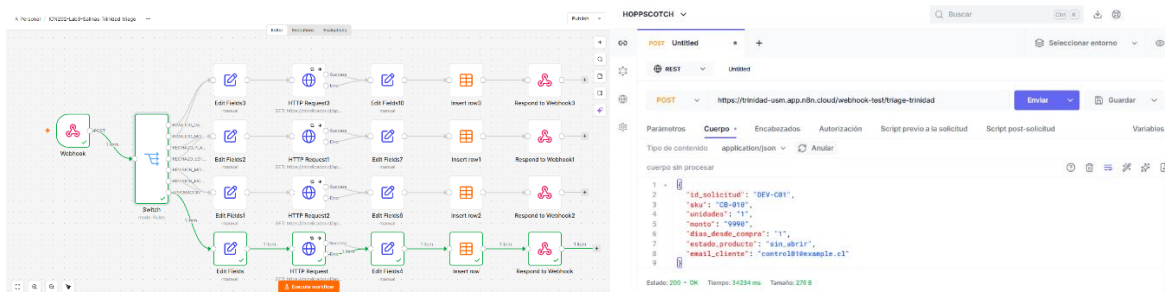
En conclusión, ambos casos confirman que las reglas establecidas están bien implementadas como “inclusivas”: un valor exactamente igual al umbral (U o D) no cae en revisión ni rechazo.

5. Cambio de tipo de una condición (Number a String)

Para este punto se tomó la Regla 2 del Switch (INVALIDO_MONTO: *monto.toNumber()* <= 0) y se cambió el tipo de la comparación de Number a String, a propósito, para ver que ocurría.



Con este cambio, se mandó la solicitud de control DEV-C01 (*monto* = \$9.990) que se encuentra en el archivo *solicitudes.json*. El resultado fue que la solicitud cayó en APROBACION, cuando debería haber caído en INVALIDO_MONTO.

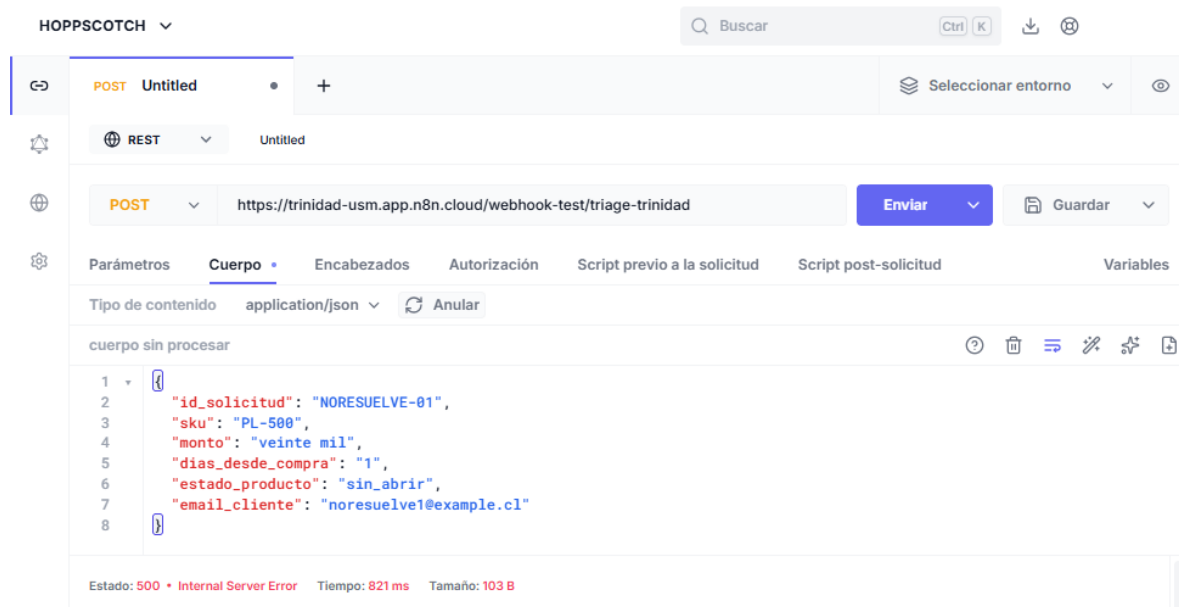


Esto ocurre ya que al cambiar el tipo a String, la comparación deja de evaluar magnitud numérica y en cambio compara el texto directamente, así que la condición deja de funcionar como estaba pensada. Esto hace que la regla se “salte” sin evaluar correctamente el monto, y la solicitud termina cayendo en APROBACION por defecto.

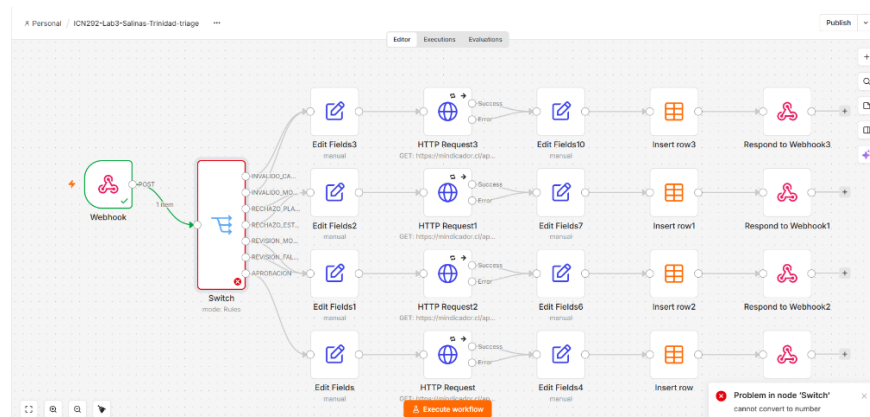
6. Dos casos que el flujo no puede resolver

Se mandaron 2 solicitudes a propósito para mostrar los límites reales del flujo.

Caso A – Monto escrito en palabras:



En este caso el nodo Switch falló directamente con el error “Cannot convert to number” y la respuesta fue un 500. Esto ocurre porque el flujo construido solo sabe interpretar montos escritos en formato numérico como texto, no en palabras. Como el Switch usa `.toNumber()` para convertir el monto, “veinte mil” no se puede convertir y el nodo falla, deteniendo toda la ejecución. Para poder detectar este caso, se necesitaría agregar una validación antes del Switch que compruebe si el monto es realmente numérico, y si no lo es, mandarlo por una ruta de invalido en vez de dejar que el nodo se caiga.



Caso B -SKU que no existe:

HOPPSHOTCH

POST Untitled

REST

POST https://trinidad-usm.app.n8n.cloud/webhook-test/triage-trinidad

Enviar Guardar

Parámetros Cuerpo Encabezados Autorización Script previo a la solicitud Script post-solicitud Variables

Tipo de contenido application/json Anular

cuerpo sin procesar

```
1 {
2   "id_solicitud": "NORESUELVE-02",
3   "sku": "ZZ-999",
4   "monto": "20000",
5   "dias_desde_compra": "1",
6   "estado_producto": "sin_abrir",
7   "email_cliente": "noresuelve2@example.cl"
8 }
```

Estado: 200 • OK Tiempo: 32890 ms Tamaño: 283 B

En este caso pasó todas las reglas sin ningún problema y terminó en APROBACION, con respuesta 200 OK. Esto ocurre ya que ninguna de las 6 reglas establecidas revisa si el SKU existe realmente en un catálogo de productos, así que, aunque “ZZ-999” no sea un producto valido, la solicitud igual se aprueba. Para detectar este caso, se necesitaría agregar una regla que compare el SKU recibido contra un catálogo real de productos validos y rechazar o marcar como invalida cualquier solicitud cuyo SKU no aparezca ahí.

